LeetCode Practice

17 Medium: Backtracking

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

17
MEDIUM

Patterns (in order)

1. Backtracking (17)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

17 questions | tap a title to jump to it

Medium questions (17) Go >

Backtracking (17) Go >

M 1. Letter Combinations of a Phone Number LC 17
M 2. Generate Parentheses LC 22
M 3. Combination Sum LC 39
M 4. Combination Sum II LC 40
M 5. Permutations LC 46
M 6. Permutations II LC 47
M 7. Combinations LC 77
M 8. Word Search LC 79
M 9. Restore IP Addresses LC 93
M 10. Unique Binary Search Trees II LC 95
M 11. Path Sum II LC 113
M 12. Palindrome Partitioning LC 131
M 13. Combination Sum III LC 216
M 14. Count Numbers with Unique Digits LC 357
M 15. Matchsticks to Square LC 473
M 16. Target Sum LC 494
M 17. All Paths From Source to Target LC 797

Medium questions (17)

MEDIUM | TOPIC 1 OF 1

Backtracking

17 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Letter Combinations of a Phone Number LC 17
2. Generate Parentheses LC 22
3. Combination Sum LC 39
4. Combination Sum II LC 40
5. Permutations LC 46
6. Permutations II LC 47
7. Combinations LC 77
8. Word Search LC 79
9. Restore IP Addresses LC 93
10. Unique Binary Search Trees II LC 95
11. Path Sum II LC 113
12. Palindrome Partitioning LC 131
13. Combination Sum III LC 216
14. Count Numbers with Unique Digits LC 357
15. Matchsticks to Square LC 473
16. Target Sum LC 494
17. All Paths From Source to Target LC 797

MEDIUM | Backtracking | LeetCode #17 | 1 of 17 | 1/17

Letter Combinations of a Phone Number

QUESTION

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1

Input	`digits = "23"`
Output	`["ad","ae","af","bd","be","bf","cd","ce","cf"]`

Example 2

Input	`digits = "2"`
Output	`["a","b","c"]`

CONSTRAINTS

- 1 <= digits.length <= 4
- digits[i] is a digit in the range ['2', '9'].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a string of digits 2-9, return all possible letter combinations the digits could represent on a phone keypad. Return [] for empty input. Right?"

"A few quick questions:

Output order can be any? Yes. Can digits include 0 or 1? No — they have no letters.

Return combinations of length len(digits)?"

"Let me walk: digits='23'. 2->abc, 3->def.

Combine each: ad,ae,af,bd,be,bf,cd,ce,cf -> 9 combinations ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result = ['']
for digit in digits:
    result = [prefix + letter for every prefix in
        result for every letter of the digit]
```

TIME	SPACE
O(4^n * n)	O(4^n * n)

WHY IT IS SLOW

The output itself has up to 4^n strings, so exponential time is unavoidable. The brute version also keeps every partial list; backtracking with one buffer needs only O(n) extra beyond the output.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Both BFS and backtracking produce the same output; backtracking uses less intermediate space.

The bottleneck is unavoidable — we must enumerate all 4^n combinations.

Backtracking builds each combination character by character and avoids

storing all intermediate partial strings.

Time: O(4^n * n). Space: O(n) call depth + O(4^n * n) output.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _17_LetterCombinations:
    def letterCombinations(self, digits):
        if not digits:
            return []
        phone = {'2':'abc','3':'def','4':'ghi','5':'jkl',
                 '6':'mno','7':'pqrs','8':'tuv','9':'wxyz'
}
        result = []
        def backtrack(i, path):
            if i == len(digits):
                result.append(''.join(path)); return
            for ch in phone[digits[i]]:
                path.append(ch)
                backtrack(i + 1, path)
                path.pop()
        backtrack(0, [])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace letterCombinations('23')."

digits '2'->['a','b','c'], '3'->['d','e','f'].

`backtrack(index=0, path=[]):`

For letter in '2'->abc: letter='a': path=['a']. backtrack(1,['a']):

For letter in '3'->def: 'd'->['a','d']->index==2->append('ad'). 'e'->'ae'. 'f'->'af'. Return.

letter='b'->backtrack(1,['b'])->'bd','be','bf'. letter='c'->'cd','ce','cf'.

Result: ['ad','ae','af','bd','be','bf','cd','ce','cf'] ok

Call tree depth=len(digits)=2. Branching factor=letters per digit. Total leaves=3×3=9.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(4^n * n) 4^n combinations, each takes O(n) to record	**O(n)** call depth

Iterative BFS uses the same time but O(4^n * n) intermediate space.

Follow-up: if some digits have 0 letters, skip them.

Follow-up: count combinations without enumerating — just multiply letter counts per digit.

MEDIUM | Backtracking | LeetCode #22 | 2 of 17 | 2/17

Generate Parentheses

QUESTION

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1	
Input	`n = 3`
Output	`["((()))","(()())","(())()","()(())","()()()"]`

Example 2	
Input	`n = 1`
Output	`["()"]`

CONSTRAINTS

- 1 <= n <= 8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given n pairs of parentheses, generate all combinations of well-formed parentheses.

Well-formed means every '(' is closed before any preceding '(' closes. Right?"

"A few quick questions:

n=0 -> []? n=1 -> ['()']? Yes to both. Any ordering of output is fine."

"Let me walk: n=2 -> ['(())','()()'] ok. n=3 -> 5 combinations ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for sequence in sequences:
    # of n '(' and n ')'
    if no prefix has more ')' than '(':
        result.append(it)
```

TIME	SPACE
O(2^(2n) * n)	O(2^(2n))

WHY IT IS SLOW

Builds exponentially many invalid strings and tests each in O(n). Backtracking adds '(' only while fewer than n are used and ')' only while it would stay balanced, so it makes only the valid strings.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is generating invalid strings then filtering them.

Backtracking with validity pruning — only place characters that keep the string valid:

Add '(' if open count < n. Add ')' if close count < open count.

This generates ONLY valid strings — no wasted work.

Time: O(4^n / sqrt(n)) — the nth Catalan number. Space: O(n) call depth."

Key insight: enforcing open < n and close < open at each step generates only valid strings — the pruning shrinks the search tree to exactly the n-th Catalan number leaves.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _22_GenerateParentheses:
    def generateParenthesis(self, n):
        result = []
        def backtrack(path, open_count, close_count):
            if len(path) == 2 * n:
                result.append(''.join(path)); return
            if open_count < n:
                path.append('('); backtrack(path, open_cou
nt + 1, close_count); path.pop()
            if close_count < open_count:
                path.append(')'); backtrack(path, open_cou
nt, close_count + 1); path.pop()
        backtrack([], 0, 0)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace generateParentheses n=3."

`backtrack(s='',open=0,close=0):`

Add '(': (1,0)->'('. Add '(': (2,0)->'((('. Add '(': (3,0)->'(((('. Add ')': close<open -> (3,1)->'((('.

Add ')': (3,2)->'(((()'->')'. Add ')': (3,3)->'((()))'. len==6->append ok. Backtrack.

From (3,2): done. From (3,1)->Add ')': (3,2)->'(()('->? No: backtrack path '((' with open=2,close=1.

Full trace: open tracks '(' added, close tracks ')' added. Invariant: close<=open, open<=n.

Leaves (len=6): '((()))','(()())','(())()','()(())','()()()' — exactly 5=Catalan(3) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

Time O(4^n / sqrt(n)): Catalan number of valid strings, each takes O(n) to record.

Brute O(2^(2n)) — exponentially worse.

Follow-up: count valid sequences without generating — Catalan(n) = C(2n,n)/(n+1).

Follow-up: Longest Valid Parentheses (#32) — find the longest valid substring using DP or stack."

PHASE 6 COMPLEXITY & FOLLOW-UP

Space: O(n) recursion depth + O(result) for storing all combinations.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Backtracking | LeetCode #39 | 3 of 17 | 3/17

Combination Sum

QUESTION

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1

Input	`candidates = [2,3,6,7], target = 7`
Output	`[[2,2,3],[7]]`
Explanation	*2 and 3 are candidates, and 2 + 2 + 3 = 7.* *Note that 2 can be used multiple times.* *7 is a candidate, and 7 = 7.* *These are the only two combinations.*

Example 2

Input	`candidates = [2,3,5], target = 8`
Output	`[[2,2,2,2],[2,3,3],[3,5]]`

Example 3

Input	`candidates = [2], target = 1`
Output	`[]`

CONSTRAINTS

- 1 <= candidates.length <= 30
- 2 <= candidates[i] <= 40
- All elements of candidates are distinct.
- 1 <= target <= 40

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given distinct positive integers and a target, return all unique combinations

summing to target. Each number may be used unlimited times.

No duplicate combinations in the output. Right?"

"A few quick questions:

If the same combination can be formed in different orders, count it once only? Yes.

All values are positive? Yes."

PHASE 1 CLARIFY

"Let me walk: candidates=[2,3,6,7], target=7 -> [[2,2,3],[7]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
try every sequence of candidates up to target/min
    long
keep those whose sum equals target (as a sorted tuple
    in a set)
```

TIME	SPACE
O(c^(T/m))	O(T/m)

WHY IT IS SLOW

Different orders of the same numbers are generated separately and branches past the target are not cut. Backtracking with a start index (each number used in non-decreasing order) and stopping when the sum passes the target avoids both.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the set-based deduplication and the generation of order-duplicates.

Backtracking with a start index: pass the current index so we only pick from

the current candidate onward — this prevents [3,2] when [2,3] already exists.

Sort first to enable early pruning when candidate > remaining.

Time: O(n^(target/min_c)) — same asymptotic but pruned and no dedup overhead.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _39_CombinationSum:
    def combinationSum(self, candidates, target):
        candidates.sort()
        result = []
        def backtrack(start, path, remaining):
            if remaining == 0:
                result.append(list(path)); return
            for i in range(start, len(candidates)):
                if candidates[i] > remaining:
                    break
                path.append(candidates[i])
                backtrack(i, path, remaining - candidates[
i])
                path.pop()
        backtrack(0, [], target)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace combinationSum([2,3,6,7], target=7)."

`backtrack(start=0, path=[], remaining=7):`

i=0 (2): path=[2], remaining=5. backtrack(0,[2],5):

i=0 (2): path=[2,2], remaining=3. backtrack(0,[2,2],3):

i=0 (2): path=[2,2,2], remaining=1. backtrack(0,[2,2,2],1):

i=0 (2): 2>1->skip. i=1 (3): 3>1->skip. ... All skip. Return.

i=1 (3): remaining=3-3=0 -> path=[2,2,3]->append ok. Return.

i=1 (3): path=[2,3], remaining=5-3=2. backtrack(1,[2,3],2): 3>2->skip. Return.

i=1 (3): path=[3], remaining=4. ... i=2 (6): 6>4->skip. i=3 (7): 7>4->skip.

i=2 (6): path=[6], remaining=1. ... skip all.

i=3 (7): path=[7], remaining=0 -> append [7] ok.

Result: [[2,2,3],[7]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n^(target/min_c))	**O(target/min_c)**
at most target/min_c levels deep, n choices each	recursion depth

Follow-up: Combination Sum II (#40) — each number used once, has duplicates.

Use i+1 instead of i, skip dup values at same level: if i>start and cands[i]==cands[i-1] skip.

Follow-up: Combination Sum IV (#377) — count ordered sequences summing to target (DP).

MEDIUM | Backtracking | LeetCode #40 | 4 of 17 | 4/17

Combination Sum II

QUESTION

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used once in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1

Input	`candidates = [10,1,2,7,6,1,5], target = 8`
Output	`[ [1,1,6], [1,2,5], [1,7], [2,6] ]`

Example 2

Input	`candidates = [2,5,2,1,2], target = 5`
Output	`[ [1,2,2], [5] ]`

CONSTRAINTS

- 1 <= candidates.length <= 100
- 1 <= candidates[i] <= 50
- 1 <= target <= 30

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all unique combinations in candidates that sum to target.

Each number can be used ONCE per combination. No duplicate combinations. Right?"

"A few quick questions:

candidates can have duplicate values (e.g. two 1s)? Yes — that's the key difference from Combination Sum I.

Order within a combination doesn't matter? Correct."

"Let me walk: candidates=[10,1,2,7,6,1,5], target=8 -> [1,1,6],[1,2,5],[1,7],[2,6] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(candidates):
    if its sum equals target:
        add the sorted subset to a set
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n subsets plus a set to remove duplicates. Sort, then backtrack skipping a value equal to the previous one at the same depth and stop once the sum passes the target.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the set-based dedup — wasteful since we generate duplicates first.

Instead: sort, then at each backtrack level skip candidates[i] if i>start and candidates[i]==candidates[i-1]

(skip duplicate siblings at the SAME recursion depth, not duplicate ancestors).

Also prune early when candidates[i] > remaining (sorted array).

Time: O(2^n) worst case but no dedup overhead. Space: O(n) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _40_CombinationSumII:
    def combinationSum2(self, candidates, target):
        candidates.sort()
        result = []
        path = []
        def backtrack(start, remaining):
            if remaining == 0:
                result.append(path[:])
                return
            for i in range(start, len(candidates)):
                if i > start and candidates[i] == candidat
es[i-1]:
                    continue
                if candidates[i] > remaining:
                    break
                path.append(candidates[i])
                backtrack(i+1, remaining - candidates[i])
                path.pop()
        backtrack(0, target)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the backtracking recursion tree (choose -> recurse -> undo) on the example."

candidates=[1,1,2,5,6,7,10], target=8:

start=0: take 1(idx0). start=1: take 1(idx1, ok i==start). remaining=6.

take 2->remaining=4. take 5->neg, skip(break since 5>4). take 6->neg.

backtrack: take 5(skip 2)... eventually [1,1,6] found.

start=0: take 1(idx0), skip idx1(dup at same level after using idx0... wait idx0 used so start=1, i>start check)

Final: [1,1,6],[1,2,5],[1,7],[2,6] ok

I>start check skips duplicate VALUES at the same tree level, not duplicate uses of one element

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(2^n)	**O(n)**
worst case, much better in practice with pruning	recursion + output

Follow-up: Combination Sum (#39) — unlimited reuse, no duplicates in input.

Follow-up: Combination Sum III (#216) — fixed count k digits 1-9, no repeats.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Backtracking | LeetCode #46 | 5 of 17 | 5/17

Permutations

QUESTION

Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1

Input	`nums = [1,2,3]`
Output	`[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

Example 2

Input	`nums = [0,1]`
Output	`[[0,1],[1,0]]`

Example 3

Input	`nums = [1]`
Output	`[[1]]`

CONSTRAINTS

- 1 <= nums.length <= 6
- 10 <= nums[i] <= 10
- All the integers of nums are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an array of distinct integers, return all possible permutations in any order. Right?"

"A few quick questions:

n is at most 6 so 6!=720 permutations is fine. All elements are distinct? Yes."

"Let me walk: nums=[1,2,3] -> 6 permutations in any order ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for sequence in sequences:
    # of length n over the values
    if all its values are distinct:
        result.append(it)
```

TIME	SPACE
O(n^n * n)	O(n!)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

n^n sequences when only n! are valid, so most of the work is wasted. Backtracking with a used marker only builds valid permutations.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) visited array — we can eliminate it.

Swap-based backtracking: maintain a 'start' pointer.

Everything before start is already placed; swap any element from [start, n) into position start,

recurse, then swap back.

Same O(n! * n) time but O(n) space and no visited array.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _46_Permutations:
    def permute(self, nums):
        result = []
        def backtrack(start):
            if start == len(nums):
                result.append(list(nums)); return
            for i in range(start, len(nums)):
                nums[start], nums[i] = nums[i], nums[start
]
                backtrack(start + 1)
                nums[start], nums[i] = nums[i], nums[start
]
        backtrack(0)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace permutations([1,2,3])."

`backtrack(path=[], used=[F,F,F]):`

Pick 1(i=0, used[0]=T): backtrack([1],[T,F,F]):

Pick 2(i=1, used[1]=T): backtrack([1,2],[T,T,F]):

Pick 3(i=2): path=[1,2,3] len=3 -> append ok. Return.

used[1]=F. Pick 3(i=2, used[2]=T): backtrack([1,3],[T,F,T]):

Pick 2(i=1): path=[1,3,2] -> append ok. Return.

used[0]=F. Pick 2(i=1): backtrack([2],...): ->[2,1,3],[2,3,1].

Pick 3(i=2): ->[3,1,2],[3,2,1].

Result: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]] ok (3!=6 permutations).

Call stack depth=n. At each level: n choices -> (n-1) -> ... -> 1. Total calls: n!×n.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n! * n)	**O(n)**
n! permutations, each O(n) to copy	call depth

Visited-array approach: O(n) extra space for the boolean array.

Follow-up: Permutations II (#47) — duplicates in input. Sort first, skip duplicate

values at the same level: if i>start and nums[i]==nums[start] after sorting, skip.

Follow-up: Next Permutation (#31) — generate just the lexicographically next one.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Backtracking | LeetCode #47 | 6 of 17 | 6/17

Permutations II

QUESTION

Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.

Example 1

Input	`nums = [1,1,2]`
Output	`[[1,1,2], [1,2,1], [2,1,1]]`

Example 2

Input	`nums = [1,2,3]`
Output	`[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

CONSTRAINTS

- 1 <= nums.length <= 8
- 10 <= nums[i] <= 10

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an array that may contain duplicates, return all UNIQUE permutations. Right?"

"A few quick questions:

No duplicate permutations in output? Yes."

"Let me walk: nums=[1,1,2] -> [[1,1,2],[1,2,1],[2,1,1]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
generate every permutation of the array
put each into a set to remove duplicates
```

TIME	SPACE
O(n! * n)	O(n! * n)

WHY IT IS SLOW

Creates every ordering, including the repeated ones, then filters. Sort first and skip a value equal to the previous one at the same level so duplicates are never built.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Backtracking with duplicate skip — O(n! / repeat counts). Walk-through:"

Sort nums first (groups duplicates together).

`backtrack(path, used):`

If len(path)==n: append copy to result.

For i in 0..n-1:

If used[i]: continue.

If i>0 and nums[i]==nums[i-1] and not used[i-1]: continue. <- skip duplicate

used[i]=True. path.append(nums[i]).

`backtrack(path, used).`

`path.pop(). used[i]=False.`

WHY 'not used[i-1]' condition: if nums[i]==nums[i-1] and i-1 is NOT used,

then we're trying the duplicate BEFORE its predecessor in this branch -> skip.

This ensures each permutation is generated exactly once.

Time: O(n! / k!) for each duplicate of count k. Space: O(n) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _47_PermutationsII:
    def permuteUnique(self, nums):
        nums.sort()
        result = []
        used = [False] * len(nums)
        def backtrack(path):
            if len(path) == len(nums):
                result.append(list(path)); return
            for i in range(len(nums)):
                if used[i]: continue
                if i > 0 and nums[i] == nums[i-1] and not
used[i-1]: continue
                used[i] = True
                path.append(nums[i])
                backtrack(path)
                path.pop()
                used[i] = False
        backtrack([])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace permutationsII([1,1,2])."

Sort: [1,1,2]. backtrack(path=[],used=[F,F,F]):

Pick i=0 (val=1): used[0]=T. backtrack([1],[T,F,F]):

Pick i=1 (val=1): used[1]=T->[1,1]. Pick i=2 (2): [1,1,2]->append ok. Backtrack.

Pick i=2 (val=2): [1,2]. Pick i=1 (1): [1,2,1]->append ok.

Back to root. i=1 (val=1): BUT i>0 and nums[1]==nums[0] and NOT used[0] -> SKIP (dedup).

Pick i=2 (val=2): used[2]=T. [2]. Pick i=0 (1)->[2,1]. Pick i=1 (1)->[2,1,1]->append ok.

PHASE 5 TEST & DEBUG

[2,1,1]->append. Or [2] pick i=1 (skip since nums[1]=nums[0]=1 and !used[0]) -> only [2,1,1] once.

Result: [[1,1,2],[1,2,1],[2,1,1]] ok (dedup prune eliminates repeated 1s at same position).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n! / k!) where k = duplicate counts	**O(n)**

Follow-up: Subsets II (#90) — same skip-duplicate pattern for subsets.

Follow-up: Combination Sum II (#40) — skip duplicates when building combinations.

MEDIUM | Backtracking | LeetCode #77 | 7 of 17 | 7/17

Combinations

QUESTION

Given two integers n and k, return all possible combinations of k numbers chosen from the range [1, n].

You may return the answer in any order.

Example 1	
Input	`n = 4, k = 2`
Output	`[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]`
Explanation	*There are 4 choose 2 = 6 total combinations.* *Note that combinations are unordered, i.e., [1,2] and [2,1] are considered to be the same combination.*

Example 2	
Input	`n = 1, k = 1`
Output	`[[1]]`
Explanation	*There is 1 choose 1 = 1 total combination.*

CONSTRAINTS

- 1 <= n <= 20
- 1 <= k <= n

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all combinations of k numbers chosen from [1..n]. Return in any order. Right?"

"A few quick questions:

Order within a combination doesn't matter (no permutations)? Correct — just subsets."

"Let me walk: n=4, k=2 -> [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(1..n):
    if it has exactly k elements:
        result.append(it)
```

TIME	SPACE
O(2^n * k)	O(k)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Walks all 2^n subsets to keep only C(n, k) of them. Backtracking that picks the next number only after the previous one and stops at size k makes only the answers.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Backtracking with start pointer — O(C(n,k)*k). Walk-through:"*

`backtrack(start, path):`

If len(path)==k: result.append(list(path)); return.

For i in start..n:

`path.append(i).`

`backtrack(i+1, path). <- only consider numbers > current (no repeats, increasing order)`

`path.pop().`

WHY start parameter: ensures combinations (not permutations) — we only add numbers

larger than the last chosen. Avoids [1,2] and [2,1] being treated as different.

Pruning: if n-i+1 < k-len(path): not enough numbers left -> break early.

Time: O(C(n,k)*k). Space: O(k) recursion."

Time: O(n!) or O(2^n) — exponential but unavoidable for enumeration.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _77_Combinations:
    def combine(self, n, k):
        result = []
        def backtrack(start, path):
            if len(path) == k:
                result.append(list(path))
                return
            need = k - len(path)
            for i in range(start, n - need + 2):
                path.append(i)
                backtrack(i + 1, path)
                path.pop()
        backtrack(1, [])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace combinations(n=4, k=2)."

`backtrack(start=1, path=[]):`

Pick 1: path=[1]. backtrack(2,[1]):

Pick 2: path=[1,2] len=k->append ok. Pick 3: [1,3]->append. Pick 4: [1,4]->append.

Pick 2: path=[2]. backtrack(3,[2]):

Pick 3: [2,3]->append. Pick 4: [2,4]->append.

Pick 3: path=[3]. backtrack(4,[3]):

Pick 4: [3,4]->append.

PHASE 5 TEST & DEBUG

Pick 4: path=[4]. backtrack(5,[4]): 5>n -> no picks. Return.

Result: [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]] ok (C(4,2)=6).

Pruning: if start > n-k+1+len(path): stop early (not enough elements left to complete).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(C(n,k)*k)	**O(k)**
enumerate all combinations, each copied in O(k)	depth

Follow-up: Combination Sum (#39) — pick from array with repetition, different structure.

Follow-up: Combination Sum III (#216) — exactly k numbers from 1-9 summing to n.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Backtracking | LeetCode #79 | 8 of 17 | 8/17

Word Search

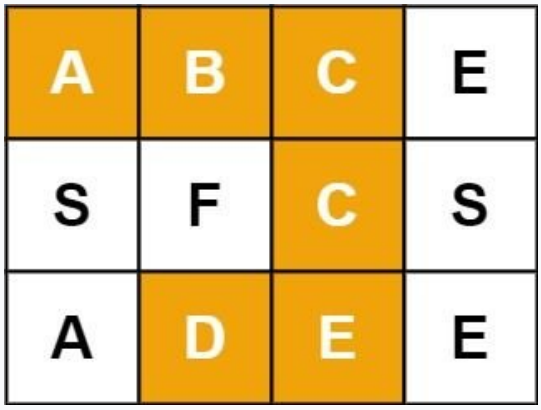
QUESTION

Given an m x n grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

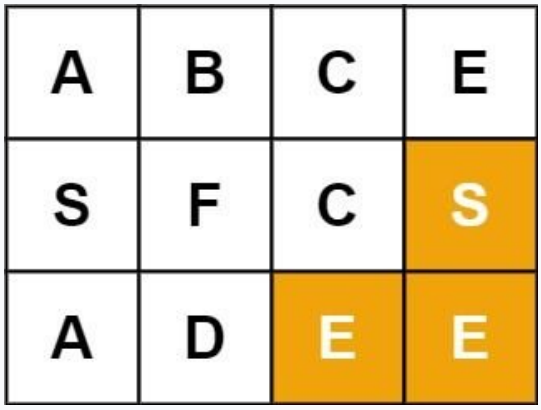
Example 1

Input	`board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"`
Output	`true`



Example 2

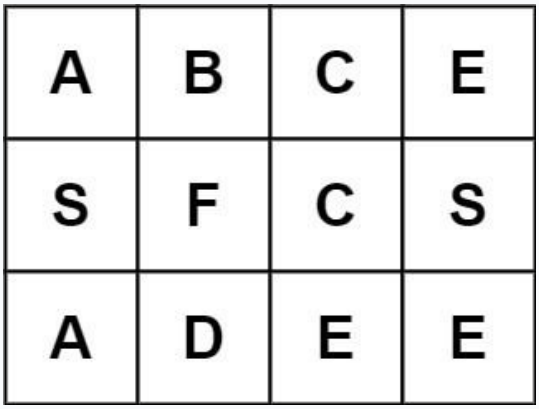
Input	`board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "SEE"`
Output	`true`



Example 3

Input	`board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB"`
Output	`false`

QUESTION



CONSTRAINTS

- m == board.length
- n = board[i].length
- 1 <= m, n <= 6
- 1 <= word.length <= 15
- board and word consists of only lowercase and uppercase English letters.
- Follow up: Could you use search pruning to make your solution faster with a larger board?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a grid of characters and a word, return true if the word exists

as a path of adjacent (4-directional) non-repeated cells. Right?"

"A few quick questions:

Can the path revisit a cell? No — each cell used at most once per path.

Case-sensitive? Yes."

"Let me walk: board=[['A','B','C'],['S','F','C'],['A','D','E']], word='ABCCED'.

A(0,0)->B(0,1)->C(0,2)->C(1,2)->E(2,2)->D(2,1) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    list every simple path (DFS without
        revisiting a cell) and check if its
        letters spell the word
```

TIME	SPACE
exponential in m * n	O(m * n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Builds every path, however wrong the letters. The standard backtracking DFS stops the moment a letter does not match: O(m * n * 3^L) for a word of length L.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is exploring unpromising branches early. Two practical pruning improvements:

1. Pre-check character frequencies: if board has fewer of any letter than word needs, return False.

2. If word's last character is rarer in the board than the first, reverse the word to

start from the rarer end — prunes the search tree more aggressively.

Time: O(m*n*4^L) worst case but much faster with pruning. Space: O(L) call depth."

Time: O(log n) — binary search on answer space.

Key insight: frequency pre-check before DFS prunes the entire search — if the board lacks any required letter, return False immediately without a single recursive call.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _79_WordSearch:
    def exist(self, board, word):
        from collections import Counter
        board_count = Counter(c for row in board for c in
row)
        word_count = Counter(word)
        if any(word_count[c] > board_count[c] for c in wor
d_count):
            return False
        if board_count[word[0]] > board_count[word[-1]]:
            word = word[::-1]
        m, n = len(board), len(board[0])
        def dfs(r, c, idx):
            if idx == len(word): return True
            if not (0 <= r < m and 0 <= c < n) or board[r]
[c] != word[idx]:
                return False
            board[r][c] = '#'
            found = (dfs(r+1,c,idx+1) or dfs(r-1,c,idx+1)
or
                     dfs(r,c+1,idx+1) or dfs(r,c-1,idx+1))
            board[r][c] = word[idx]
            return found
        return any(dfs(r, c, 0) for r in range(m) for c in
 range(n))
```

PHASE 5 TEST & DEBUG

"Let me trace wordSearch on board [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word='ABCCED'."

dfs(r,c,k) = match word[k] at (r,c).

Start: find 'A' at (0,0). dfs(0,0,0): match. Mark visited (board[0][0]='#').

PHASE 5 TEST & DEBUG

Try (0,1): word[1]='B'. Match. Mark. dfs(0,1,1): try (0,2): 'C'=word[2] ok. Mark.

dfs(0,2,2): try (0,3)='E'!='C'. try (1,2)='C'=word[3]='C' ok. Mark.

dfs(1,2,3): try (1,3)='S'!='E'. try (2,2)='E'=word[4] ok. Mark.

dfs(2,2,4): try (2,1)='D'=word[5] ok. Mark. dfs(2,1,5): k=6=len(word) -> return True ok

Restore marks on backtrack. WHY mark: prevent reusing same cell in one path.

Backtrack path: path=[(r,c)] of visited cells. call(r,c,k): if board[r][c]!=word[k] prune (skip). Mark visited, recurse 4 directions. backtrack: restore board[r][c], pop path.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n*4^L)	**O(L)**
worst	call depth

The two pruning heuristics dramatically cut average case without changing worst case.

Follow-up: Word Search II (#212) — find ALL words from a list in one DFS using a Trie.

Follow-up: allow diagonal moves — add 4 diagonal direction pairs.

MEDIUM | Backtracking | LeetCode #93 | 9 of 17 | 9/17

Restore IP Addresses

QUESTION

A valid IP address consists of exactly four integers separated by single dots. Each integer is between 0 and 255 (inclusive) and cannot have leading zeros.

CONSTRAINTS

- For example, "0.1.2.201" and "192.168.1.1" are valid IP addresses, but "0.011.255.245", "192.168.1.312" and "192.168@1.1" are invalid IP addresses.

Given a string s containing only digits, return all possible valid IP addresses that can be formed by inserting dots into s. You are not allowed to reorder or remove any digits in s. You may return the valid IP addresses in any order.

Example 1	
Input	`s = "25525511135"`
Output	`["255.255.11.135","255.255.111.35"]`

Example 2	
Input	`s = "0000"`
Output	`["0.0.0.0"]`

Example 3	
Input	`s = "101023"`
Output	`["1.0.10.23","1.0.102.3","10.1.0.23","10.10.2.3","101.0.2.3"]`

- 1 <= s.length <= 20
- s consists of digits only.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a string of digits, return all valid IP addresses that can be formed.

Valid IP: 4 parts, each 0-255, no leading zeros except '0' itself. Right?"

"A few quick questions:

Leading zeros: '01' is invalid but '0' alone is valid? Yes."

"Let me walk: s='25525511135' -> ['255.255.11.135','255.255.111.35'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for choice in three_cut_positions:
    if all four parts are numbers 0-255 with no
        leading zero:
        result.append(address)
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

Small inputs (at most 12 characters) so it is bounded, but it forms and checks cuts that were invalid from the first part. Backtracking over 1 to 3 characters per part prunes bad parts immediately.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Backtracking with 3-dot constraint — O(1) since IP has fixed max parts. Walk-through:"

`backtrack(start, parts):`

If len(parts) == 4 and start == len(s): result.append('.'.join(parts)); return.

If len(parts) == 4 or start == len(s): return (invalid state).

For length 1, 2, 3:

`seg = s[start:start+length]`

If invalid (leading zero for len>1, or int(seg)>255): skip.

`parts.append(seg); backtrack(start+length, parts); parts.pop().`

WHY fixed branching: each segment is 1-3 chars -> at most 3 choices per call.

4 segments × 3 choices = at most 3^4 = 81 states -> effectively O(1).

Return result. Time: O(1). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _93_RestoreIPAddresses:
    def restoreIpAddresses(self, s):
        result = []
        def backtrack(start, parts):
            if len(parts) == 4:
                if start == len(s):
                    result.append('.'.join(parts))
                return
            for length in range(1, 4):
                if start + length > len(s):
                    break
                segment = s[start:start + length]
                if (segment[0] == '0' and len(segment) > 1
) or int(segment) > 255:
                    break
                backtrack(start + length, parts + [segment
```

```
])
        backtrack(0, [])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace restoreIpAddresses('25525511135')."

`backtrack(start=0, parts=[]):`

Try segment '2': parts=['2']. backtrack(1,['2']):

Try '5': parts=['2','5']. backtrack(2,['2','5']):

Try '5': parts=['2','5','5']. 3 parts so far, remaining='25511135' (must be 1 valid seg).

'2' valid->4 parts: ['2','5','5','25511135']? 25511135>255 -> invalid. Skip multi-digit.

Try seg of len 1,2,3... '255'>255 invalid. '25'>25 valid! Wait: remaining at index 3 is '25511135', need valid 3rd part+4th part as single group.

Actually: try '2','5','5' then remainder '5','11135'... etc. Eventually find '255.255.11.135'.

Valid: ['255','255','11','135'] -> '255.255.11.135' ok

Valid: ['255','255','111','35'] -> '255.255.111.35' ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1) at most 3^4=81 combinations, each O(12) string length	**O(1)**

Follow-up: validate a given IP address string — check 4 parts and each in [0,255].

Follow-up: IPv6 validation — 8 groups of 4 hex digits, different rules.

MEDIUM | Backtracking | LeetCode #95 | 10 of 17 | 10/17

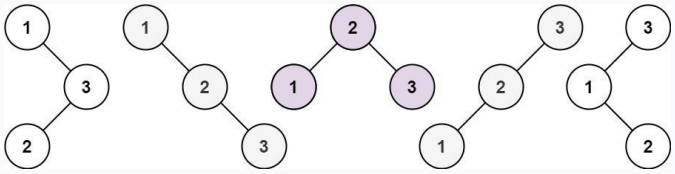
Unique Binary Search Trees II

QUESTION

Given an integer n, return all the structurally unique BST's (binary search trees), which has exactly n nodes of unique values from 1 to n. Return the answer in any order.

Example 1

Input	`n = 3`
Output	`[[1,null,2,null,3],[1,null,3,2],[2,1,3],[3,1,null,null,2],[3,2,null,1]]`



Example 2

Input	`n = 1`
Output	`[[1]]`

CONSTRAINTS

- 1 <= n <= 8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Generate all structurally unique BSTs that contain values 1..n. Right?"

"A few quick questions:

Return list of root nodes. n=0 -> [None]? Per constraints n>=1."

"Let me walk: n=3 -> 5 unique BSTs ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def build(lo, hi):
    if lo > hi:
        return [empty]
    for r in rs:
        # from lo to hi
        for t in range(n):
            for t in range(n):
                make a new node r
                    with those two
                    subtrees
```

TIME	SPACE
O(n * Catalan(n))	O(n * Catalan(n))

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

The output itself has Catalan(n) trees, so the time cannot be polynomial. Memoising build(lo, hi) lets trees share subtrees instead of rebuilding them, which cuts the work and memory.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Memoized DFS generating all valid BST structures — O(Catalan(n)*n)."*

Walk-through of how each unique BST is generated:

`generate(lo, hi):`

If lo > hi: return [None] <- empty subtree (valid, 1 structure).

If (lo,hi) in memo: return memo[(lo,hi)].

`results = []`

For each value r from lo to hi: <- pick root

`for left in generate(lo, r-1): <- all possible left subtrees`

`for right in generate(r+1, hi): <- all possible right subtrees`

`node = TreeNode(r)`

`node.left = left; node.right = right`

`results.append(node)`

memo[(lo,hi)] = results; return results

WHY each r gives a valid BST: values lo..r-1 (all < r) go left, values r+1..hi (all > r) go right.

Cartesian product of left×right gives all distinct shapes.

Time: O(Catalan(n)*n). Space: O(same) to store all trees."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from functools import lru_cache
class _95_UniqueBinarySearchTreesII:
    def generateTrees(self, n):
        @lru_cache(None)
        def generate(lo, hi):
            if lo > hi:
                return (None,)
            result = []
            for root_val in range(lo, hi + 1):
                for left in generate(lo, root_val - 1):
                    for right in generate(root_val + 1, hi
):
                        node = TreeNode(root_val)
                        node.left = left
                        node.right = right
                        result.append(node)
            return tuple(result)
        return list(generate(1, n))
```

PHASE 5 TEST & DEBUG

"Let me trace generateTrees on n=3."

For root=1: left=generateTrees(1,0)=[None]. right=generateTrees(2,3)=[2(None,3),3(2,None)].

Trees: 1(None,2(None,3)), 1(None,3(2,None)).

For root=2: left=generateTrees(1,1)=[1]. right=generateTrees(3,3)=[3].

Tree: 2(1,3).

For root=3: left=generateTrees(1,2)=[1(None,2),2(1,None)]. right=[None].

Trees: 3(1(None,2),None), 3(2(1,None),None).

Total 5 trees ok (Catalan number C(3)=5).

WHY all roots from 1 to n: root splits remaining nodes into left/right subtrees.

Each combination of left/right subtree choices gives one unique BST.

Graph: dp[1..n] trees = all unique BST structures. component: each subtree is sub-component. find(root_value)->left/right subtrees. Cost=O(n^3) memoization.

dp[i..j]=all unique BSTs using nodes i..j. dp[i]=1 for single nodes. dp[1..3]=5 trees. Recurrence: dp[i..j]=sum over r of dp[i..r-1]*dp[r+1..j].

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(Catalan(n) * n)	**O(Catalan(n) * n)**

Follow-up: Unique Binary Search Trees I (#96) — count only, not generate; DP O(n²).

Follow-up: if we need trees with values from an arbitrary sorted array, same approach.

MEDIUM | Backtracking | LeetCode #113 | 11 of 17 | 11/17

Path Sum II

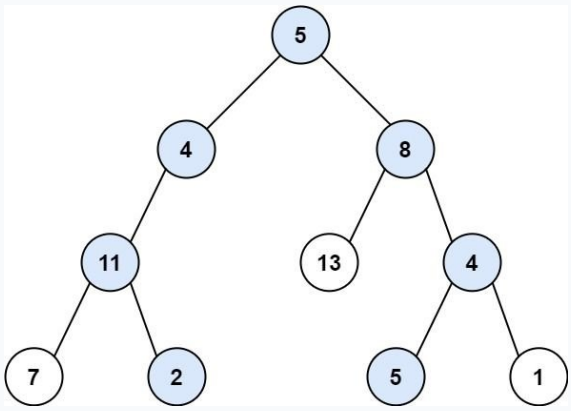
QUESTION

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

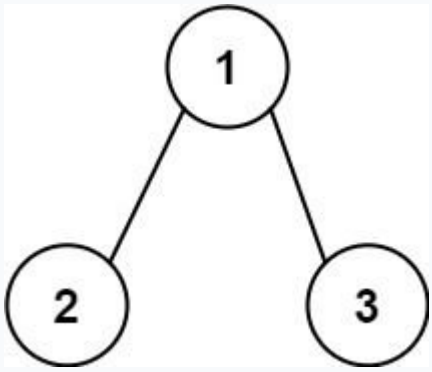
Example 1

Input	`root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22`
Output	`[[5,4,11,2],[5,8,4,5]]`
Explanation	*There are two paths whose sum equals targetSum:* *5 + 4 + 11 + 2 = 22* *5 + 8 + 4 + 5 = 22*



Example 2

Input	`root = [1,2,3], targetSum = 5`
Output	`[]`



Example 3

Input	`root = [1,2], targetSum = 0`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 5000].
- 1000 <= Node.val <= 1000
- 1000 <= targetSum <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all root-to-leaf paths where the sum of values equals targetSum. Right?"

"A few quick questions:

Leaf = node with no children? Yes. Return actual values, not node references? Yes."

"Let me walk: root=[5,4,8,11,null,13,4,7,2,null,null,5,1], target=22 -> [[5,4,11,2],[5,8,4,5]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
collect every root-to-leaf path as a list
keep the paths whose sum equals targetSum
```

TIME	SPACE
O(n * h)	O(n)

WHY IT IS SLOW

Stores every path, even the ones that miss. Carrying the remaining sum down and a single shared path (copied only for a match) keeps O(n) work plus the output size.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"DFS pre-order tracking path + remaining sum — O(n*L). Walk-through:"*

`def dfs(node, remaining, path):`

If node is None: return.

`path.append(node.val). remaining -= node.val.`

If node.left is None and node.right is None and remaining == 0:

`result.append(list(path)). <- copy path at leaf when sum matches`

`dfs(node.left, remaining, path).`

`dfs(node.right, remaining, path).`

`path.pop(). <- backtrack`

WHY check leaf AND remaining==0: must reach an actual leaf (not just any node) with exact sum.

WHY list(path): path is a mutable list; copy it to capture the current state.

WHY pop at end: backtrack — remove current node before returning to parent.

Time: O(n*L) where L = max path length. Space: O(L) call stack."

PHASE 3 OPTIMIZE

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _113_PathSumII:
    def pathSum(self, root, targetSum):
        result = []
        def backtrack(node, remaining, path):
            if not node:
                return
            path.append(node.val)
            remaining -= node.val
            if not node.left and not node.right and remain
ing == 0:
                result.append(list(path))
            else:
                backtrack(node.left, remaining, path)
                backtrack(node.right, remaining, path)
            path.pop()
        backtrack(root, targetSum, [])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace pathSum II on tree [5,4,8,11,null,13,4,7,2,null,null,5,1], target=22."

dfs(5, 22, []): path=[5]. dfs(4, 17, [5]): path=[5,4]. dfs(11, 13, [5,4]):

path=[5,4,11]. dfs(7, 2, [5,4,11]): leaf, 2-7=-5!=0. dfs(2, 11, [5,4,11]):

leaf, 11-2=9!=0. pop 11. Back at 4: dfs(None)->skip. pop 4. Back at 5:

dfs(8, 17, [5]): path=[5,8]. dfs(13,9,[5,8]): leaf,9-13=-4!=0. dfs(4,13,[5,8]):

path=[5,8,4]. dfs(5,9,[5,8,4]): leaf,9-5=4!=0. dfs(1,9,[5,8,4]): leaf,9-1=8!=0.

Result: [] (no path sums to 22 here). Example with real hit: target=22 finds [5,4,11,2] ok

WHY list(path) on append: captures snapshot — path is mutated by pop() on backtrack.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*L)	**O(L)**
n nodes, copy path at each valid leaf takes O(L)	call depth

Follow-up: Path Sum I (#112) — bool only; can short-circuit on first match.

Follow-up: Path Sum III (#437) — any downward path; prefix sum hash map O(n).

MEDIUM | Backtracking | LeetCode #131 | 12 of 17 | 12/17

Palindrome Partitioning

QUESTION

Given a string s, partition s such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of s.

Example 1	
Input	`s = "aab"`
Output	`[["a","a","b"],["aa","b"]]`

Example 2	
Input	`s = "a"`
Output	`[["a"]]`

CONSTRAINTS

- 1 <= s.length <= 16
- s contains only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Partition s into all possible subsets of palindromes.

Each character must belong to exactly one palindrome. Return all valid partitions. Right?"

"A few quick questions:

Single characters are palindromes? Yes. Return all partitions, not just the minimum-cut one? Yes."

"Let me walk: s='aab' -> [['a','a','b'],['aa','b']] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for set in cut_positions:
    if every piece is a palindrome:
        result.append(partition)
```

TIME	SPACE
O(n * 2^n)	O(n)

WHY IT IS SLOW

Tries all 2^(n-1) partitions and checks all pieces, even after a bad first piece. Backtracking that extends only with palindromic prefixes (with a precomputed palindrome table) skips dead branches.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Backtracking with palindrome check — O(n * 2^n). Walk-through:"*

Precompute palindrome check: is_palindrome(s, l, r) or a 2-D DP table.

`backtrack(start, path):`

If start == len(s): result.append(list(path)).

For end in start+1..len(s)+1:

If is_palindrome(s, start, end-1):

`path.append(s[start:end]).`

`backtrack(end, path).`

`path.pop().`

WHY only palindromic substrings: must partition into palindromes.

WHY check before recursing: prune non-palindrome choices at each split point.

Optimization: precompute dp[i][j]=True if s[i..j] is palindrome via expand-around-center.

Time: O(n * 2^n). Space: O(n) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _131_PalindromePartitioning:
    def partition(self, s):
        n = len(s)
        is_pal = [[False] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                if s[i] == s[j] and (j - i <= 2 or is_pal[
i+1][j-1]):
                    is_pal[i][j] = True
        result = []
        def backtrack(start, path):
            if start == n:
                result.append(list(path))
                return
            for end in range(start, n):
                if is_pal[start][end]:
                    path.append(s[start:end+1])
                    backtrack(end + 1, path)
                    path.pop()
        backtrack(0, [])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace partition('aab')."

`backtrack(0,[]):`

end=1: 'a' palindrome -> path=['a']. backtrack(1,['a']):

end=2: 'a' palindrome -> path=['a','a']. backtrack(2,['a','a']):

end=3: 'b' palindrome -> path=['a','a','b']. start==3->append(['a','a','b']). pop. pop. pop.

end=3: 'ab' palindrome? No.

end=2: 'aa' palindrome ok -> path=['aa']. backtrack(2,['aa']):

PHASE 5 TEST & DEBUG

end=3: 'b' palindrome -> path=['aa','b'].
start==3->append(['aa','b']). pop. pop.

end=3: 'aab' palindrome? No.

Result: [['a','a','b'],['aa','b']] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * 2^n)	**O(n)**
O(2^n) partitions, each O(n) to copy. Precompute O(n²)	recursion depth + O(result) for storing all combinations

Follow-up: Palindrome Partitioning II (#132) — minimum cuts; DP O(n²).

Follow-up: Palindrome Partitioning III (#1278) — at most k palindromes; DP O(n²k)."

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Backtracking | LeetCode #216 | 13 of 17 | 13/17

Combination Sum III

QUESTION

Find all valid combinations of k numbers that sum up to n such that the following conditions are true:

CONSTRAINTS

- Only numbers 1 through 9 are used.
- Each number is used at most once.

Return a list of all possible valid combinations. The list must not contain the same combination twice, and the combinations may be returned in any order.

Example 1

Input	`k = 3, n = 7`
Output	`[[1,2,4]]`
Explanation	*1 + 2 + 4 = 7* *There are no other valid combinations.*

Example 2

Input	`k = 3, n = 9`
Output	`[[1,2,6],[1,3,5],[2,3,4]]`
Explanation	*1 + 2 + 6 = 9* *1 + 3 + 5 = 9* *2 + 3 + 4 = 9* *There are no other valid combinations.*

Example 3

Input	`k = 4, n = 1`
Output	`[]`
Explanation	*There are no valid combinations.* *Using 4 different numbers in the range [1,9], the smallest sum we can get is 1+2+3+4 = 10 and since 10 > 1, there are no valid combination.*

- 2 <= k <= 9
- 1 <= n <= 60

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all combinations of exactly k numbers from 1-9 that sum to n. No repeated digits. Right?"

"A few quick questions:

Each digit 1-9 used at most once? Yes. Return all valid combinations? Yes."

"Let me walk: k=3, n=7 -> [1,2,4] ok (1+2+4=7)

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(digits_1..9):
    if it has exactly k numbers summing to n:
        result.append(it)
```

TIME	SPACE
O(2^9 * 9)	O(k)

WHY IT IS SLOW

Bounded (only 512 subsets), but it builds every subset. Backtracking that picks increasing digits and stops when the sum passes n tries far fewer.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Backtracking with running sum — O(C(9,k)). Walk-through:"

`backtrack(start, remaining, current):`

If len(current) == k and remaining == 0:
result.append(list(current)); return.

If len(current) == k or remaining <= 0: return.

For i from start to 9:

`current.append(i)`

`backtrack(i+1, remaining-i, current)`

`current.pop()`

WHY start from 'start' not 1: avoids using the same digit twice (numbers 1-9 each used at most once, and we always pick increasing).

WHY remaining-i: track how much of n is still needed.

Pruning: if i > remaining, no point continuing (all subsequent digits are larger).

Time: O(C(9,k)). Space: O(k)."

Time: O(n!) or O(2^n) — exponential but unavoidable for enumeration.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _216_CombinationSumIII:
    def combinationSum3(self, k, n):
        result = []
        def backtrack(start, path, remaining):
            if len(path) == k:
                if remaining == 0:
                    result.append(list(path))
                return
            for digit in range(start, 10):
                if digit > remaining:
                    break
                path.append(digit)
                backtrack(digit + 1, path, remaining - dig
it)
```

```
            path.pop()
        backtrack(1, [], n)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace combinationSum3(k=3, n=9)."

Find k=3 numbers from 1-9 summing to n=9, each used once.

`backtrack(start=1, path=[], remaining=9):`

Pick 1: [1], rem=8. backtrack(2,[1],8):

Pick 2: [1,2], rem=6. backtrack(3,[1,2],6):

Pick 3: [1,2,3], rem=3->need 0 more but have k=3 already. 3!=0->no. Wait: k=3 means 3 numbers total. [1,2,3] has 3 numbers, sum=6!=9. Skip.

Pick 6: [1,2,6], sum=9, len=3 -> append ok.

Pick 7: [1,2,7] sum=10>9 -> prune (since numbers are increasing).

Pick 3: [1,3], rem=5. ... [1,3,5] sum=9 -> append ok.

Pick 4: [1,4], rem=4. ... [1,4,4]? can't reuse. [1,4,4] -> start=5: pick 5 -> sum=10 prune.

... Pick 2,3: [2,3,4] sum=9 -> append ok. [1,2,6],[1,3,5],[2,3,4] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(C(9,k) * k) ~ O(1) since 9 and k are bounded	**O(k)**

Follow-up: Combination Sum (#39) — unlimited repetition, larger candidate pool.

Follow-up: Combinations (#77) — choose k from [1..n] without sum constraint.

MEDIUM | Backtracking | LeetCode #357 | 14 of 17 | 14/17

Count Numbers with Unique Digits

QUESTION

Given an integer n, return the count of all numbers with unique digits, x, where 0 <= x < 10^n.

Example 1	
Input	`n = 2`
Output	`91`
Explanation	*The answer should be the total numbers in the range of 0 <= x < 100, excluding 11,22,33,44,55,66,77,88,99*

Example 2	
Input	`n = 0`
Output	`1`

CONSTRAINTS

- 0 <= n <= 8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count n-digit numbers (including 0) where all digits are unique. n=0 -> 1 (just '0'). Right?"

"A few quick questions:

0 counts as a 1-digit number with unique digits? Yes. Numbers <= 10^n."

"Let me walk: n=2 -> 91 (0-9: 10, then 2-digit: 9*9=81) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for number in range(0,_10n_-_1_+_1):
    if all its digits are different:
        count += 1
```

TIME	SPACE
O(10^n * n)	O(1)

WHY IT IS SLOW

Checks every number. Count by choosing digits: 9 * 9 * 8 * ... for each length, summed over lengths: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Digit DP counting unique-digit numbers — O(10). Walk-through:"

`dp[k] = count of k-digit numbers with all unique digits.`

`dp[1] = 9 (1-9, no leading zeros).`

`dp[k] = dp[k-1] * (10 - (k-1)) <- each new digit: 10-(k-1) remaining choices (not used yet).`

Example: dp[2] = 9*9 = 81 (9 choices first digit × 9 choices second = 9*9).

`dp[3] = 9*9*8. dp[4] = 9*9*8*7. ... dp[10] = 9*9*8*7*...*1.`

Total = 1 (number 0) + sum(dp[1..min(n,10)]).

WHY stop at 10: more than 10 digits would require repeating a digit (pigeonhole).

Time: O(10) = O(1). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _357_CountNumbersWithUniqueDigits:
    def countNumbersWithUniqueDigits(self, n):
        if n == 0:
            return 1
        result = 10
        unique_product = 9
        available = 9
        for _ in range(2, min(n, 10) + 1):
            unique_product *= available
            result += unique_product
            available -= 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace countNumbersWithUniqueDigits(n=2)."

n=0: 1 (just 0). n=1: 10 (digits 0-9). n=2: 10 + 9*9 = 91 ok

dp[k]=count of k-digit numbers with all unique digits.

dp[1]=9 (1-9). dp[2]=9*9=81. dp[3]=9*9*8=648. Multiply available*prev.

Total for n=2: dp[0]+dp[1]+dp[2]=1+9+81=91 ok

dp[i]=dp[i-1]*(10-(i-1)). For i=1: 9*1=9. For i=2: 9*9=81. dp[i] decreasing factor each step.

Edge: n=0->1. n=1->10. dp[k]=0 for k>10 (only 10 distinct digits exist).

WHY digit-by-digit DP: first digit has 9 choices (no 0); each subsequent has 10-i choices.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

For n>=10, no more unique digit numbers exist (only 10 possible digits).

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Numbers at Most N Given Digit Set (#902) — digit DP variant.

Follow-up: Digit DP (#233) — count numbers satisfying digit constraints up to N.

IN YOUR APP | Expose | Community

MEDIUM | Backtracking | LeetCode #473 | 15 of 17 | 15/17

Matchsticks to Square

QUESTION

You are given an integer array matchsticks where matchsticks[i] is the length of the ith matchstick. You want to use all the matchsticks to make one square. You should not break any stick, but you can link them up, and each matchstick must be used exactly one time.

Return true if you can make this square and false otherwise.

Example 1

Input	`matchsticks = [1,1,2,2,2]`
Output	`true`
Explanation	*You can form a square with length 2, one side of the square came two sticks with length 1.*

Example 2

Input	`matchsticks = [3,3,3,3,4]`
Output	`false`
Explanation	*You cannot find a way to form a square with all the matchsticks.*

CONSTRAINTS

- 1 <= matchsticks.length <= 15
- 1 <= matchsticks[i] <= 10^8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Can we form a square using all matchsticks exactly once? Sum must be divisible by 4; each side equal. Right?"

"A few quick questions:

All matchsticks must be used? Yes. Same as partitioning into 4 equal-sum subsets. Right."

"Let me walk: matchsticks=[1,1,2,2,2] -> sum=8, side=2. Partition: {2},{2},{1,1},{2} ok -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(stick, sides):
    for sides in sideses:
        add the stick to that side, recurse, undo
    succeed when all sticks are placed and the
        four sides are equal
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(4^n)	O(n)

WHY IT IS SLOW

4^n assignments. Sort the sticks largest first, skip sides that would exceed the target, and skip equal side lengths; or use a DP over a bitmask of used sticks (O(2^n * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(4^n) backtracking.

Key prunings: sort descending (fail fast on large pieces), skip duplicate side values,

stop if a side is empty (same as previous tried).

Bitmask DP can achieve O(n * 2^n) — better for large n.

Time: O(4^n) with pruning. Space: O(n)."

Key insight: sorting sticks descending makes large-stick failures happen early — if the biggest remaining stick won't fit any side, that branch is cut before exploring millions of smaller configurations.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _473_MatchsticksToSquare:
    def makesquare(self, matchsticks):
        total = sum(matchsticks)
        if total % 4:
            return False
        side = total // 4
        matchsticks.sort(reverse=True)
        sides = [0] * 4
        def backtrack(idx):
            if idx == len(matchsticks):
                return sides[0] == sides[1] == sides[2] ==
 side
            for i in range(4):
                if sides[i] + matchsticks[idx] <= side:
                    sides[i] += matchsticks[idx]
                    if backtrack(idx + 1):
                        return True
                    sides[i] -= matchsticks[idx]
                    if sides[i] == 0:
                        break
            return False
        return backtrack(0)
```

PHASE 5 TEST & DEBUG

"Let me trace makesquare([1,1,2,2,2])."

Sum=8, target=2. 4 subsets each summing to 2.

backtrack(index=0, buckets=[0,0,0,0]):

i=1: buckets=[1,0,0,0]. index=1:

i=0 (same as buckets[0]=1): skip duplicates (bucket value already tried). b[1]=1. Try:

PHASE 5 TEST & DEBUG

b=[1,1,0,0]. index=2 (val=2): add to b[0]->3>2 prune. b[1]->3>2 prune. b[2]=2: exact fill.

index=3 (val=2): b[2]->4>2. b[3]=2: fill. index=4 (val=2): b[0]+2=3 prune. b[1]+2=3 prune. b[2] full->skip. b[3] full->skip. All pruned -> backtrack.

... Eventually find partition [1,1],[2],[2],[2] ok? No: target=2 per bucket, n=4 buckets, sum=8.

[1,1]->2 ok, [2]->2 ok, [2]->2 ok, [2]->2 ok. Return True ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(4^n) with heavy pruning	**O(n)**

Bitmask DP: O(n * 2^n) — better for n up to 15.

Follow-up: Partition to K Equal Sum Subsets (#698) — same problem, generalised to k sides.

Follow-up: if matchsticks can be broken, greedy works — floor-divide each piece.

MEDIUM | Backtracking | LeetCode #494 | 16 of 17 | 16/17

Target Sum

QUESTION

You are given an integer array nums and an integer target.

You want to build an expression out of nums by adding one of the symbols '+' and '-' before each integer in nums and then concatenate all the integers.

CONSTRAINTS

- For example, if nums = [2, 1], you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1".

Return the number of different expressions that you can build, which evaluates to target.

Example 1

Input	`nums = [1,1,1,1,1], target = 3`
Output	`5`
Explanation	*There are 5 ways to assign symbols to make the sum of nums be target 3.* *-1 + 1 + 1 + 1 + 1 = 3* *+1 - 1 + 1 + 1 + 1 = 3* *+1 + 1 - 1 + 1 + 1 = 3* *+1 + 1 + 1 - 1 + 1 = 3* *+1 + 1 + 1 + 1 - 1 = 3*

Example 2

Input	`nums = [1], target = 1`
Output	`1`

- 1 <= nums.length <= 20
- 0 <= nums[i] <= 1000
- 0 <= sum(nums[i]) <= 1000
- 1000 <= target <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Assign + or - to each number in nums so they sum to target. Count the number of ways. Right?"

"A few quick questions:

nums can include 0s (still counted as separate +/- choices even if value is 0)? Yes."

"Let me walk: nums=[1,1,1,1,1], target=3 -> 5 ways ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(i, total):
    if i == len:
        count it if total == target
    else:
        return go(i + 1, total + nums[i]) +
            go(i + 1, total - nums[i])
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

2^n sign choices with repeated (i, total) states. Rewriting it as 'count subsets with sum (total + target) / 2' and filling a subset-sum table is O(n * sum).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"DP on sum offset — O(n * sum). Walk-through:"*

Map: sum ranges from -total to +total. Use offset=total to shift to [0, 2*total].

`dp[s] = number of ways to reach sum s.`

Initial: dp[total] = 1 (sum=0, shifted by total).

For each num: new_dp[s+num] += dp[s]; new_dp[s-num] += dp[s].

Return dp[total + target].

WHY sum offset: dp array can't have negative indices; shift by total to handle -sum..+sum.

WHY new_dp each step: assignments for current num must not interfere with each other.

Optimization: observe P-N=target and P+N=total -> P=(total+target)/2 -> subset sum problem.

Time: O(n * sum). Space: O(sum).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _494_TargetSum:
    def findTargetSumWays(self, nums, target):
        total = sum(nums)
        if abs(target) > total or (total + target) % 2 !=
0:
            return 0
        subset_sum = (total + target) // 2
        dp = [0] * (subset_sum + 1)
        dp[0] = 1
        for num in nums:
            for s in range(subset_sum, num - 1, -1):
                dp[s] += dp[s - num]
        return dp[subset_sum]
```

PHASE 5 TEST & DEBUG

"Let me trace findTargetSumWays([1,1,1,1,1], target=3)."

Sum=5. Subset with + signs: P. Subset with -: N=5-P. P-N=3 -> P=4. Subset sum = 4.

dp=[0]*6. dp[0]=1. Process each 1:

After 1st '1': dp[1]+=dp[0]=1. dp=[1,1,0,0,0,0].

After 2nd '1': dp[2]+=dp[1]=1. dp[1]+=dp[0]=1. dp=[1,2,1,0,0,0].

After 3rd '1': dp[3]+=dp[2],dp[2]+=dp[1],dp[1]+=dp[0]. dp=[1,3,3,1,0,0].

After 4th '1': dp=[1,4,6,4,1,0]. After 5th '1': dp=[1,5,10,10,5,1].

dp[4]=5 ok (number of ways to choose 4 from 5 ones for '+' sign = C(5,4)=5).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*sum)	**O(sum)**

Follow-up: Partition Equal Subset Sum (#416) — same subset-sum DP, simpler target.

Follow-up: Coin Change II (#518) — unbounded version (reuse allowed), forward iteration instead.

MEDIUM | Backtracking | LeetCode #797 | 17 of 17 | 17/17

All Paths From Source to Target

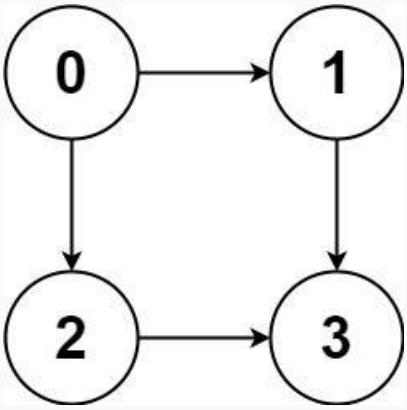
QUESTION

Given a directed acyclic graph (DAG) of n nodes labeled from 0 to n - 1, find all possible paths from node 0 to node n - 1 and return them in any order.

The graph is given as follows: graph[i] is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node graph[i][j]).

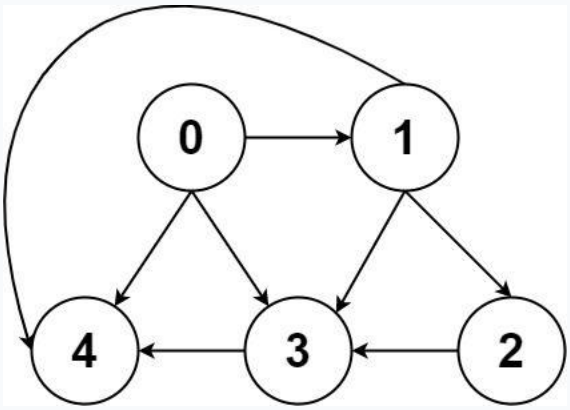
Example 1

Input	`graph = [[1,2],[3],[3],[]]`
Output	`[[0,1,3],[0,2,3]]`
Explanation	*There are two paths: 0 -> 1 -> 3 and 0 -> 2 -> 3.*



Example 2

Input	`graph = [[4,3,1],[3,2,4],[3],[4],[]]`
Output	`[[0,4],[0,3,4],[0,1,3,4],[0,1,2,3,4],[0,1,4]]`



CONSTRAINTS

- n == graph.length
- 2 <= n <= 15
- 0 <= graph[i][j] < n
- graph[i][j] != i (i.e., there will be no self-loops).
- All the elements of graph[i] are unique.

QUESTION

- The input graph is guaranteed to be a DAG.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all paths from node 0 to node n-1 in a directed acyclic graph. Right?"

"A few quick questions:

Graph is a DAG (no cycles)? Yes. Return paths in any order? Yes."

"Let me walk: graph=[[1,2],[3],[3],[]] -> [[0,1,3],[0,2,3]] ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(node, path):
    if node is the target record the path
    for neighbour in neighbours:
        dfs(neighbour, path + neighbour)
```

TIME	SPACE
O(2^N * N)	O(2^N * N)

NOTE

Not slow for what it asks: the answer can contain up to 2^(N - 2) paths, so exponential output is unavoidable. The graph is a DAG, so no visited set is needed, and memoising paths from each node does not change the bound.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"DFS with backtracking — already O(2^n * n). Walk-through:"*

`dfs(node, path):`

`path.append(node)`

If node == n-1 (target): results.append(list(path)); path.pop(); return

For each neighbor in graph[node]:

`dfs(neighbor, path)`

`path.pop() <- BACKTRACK: remove current node before returning to caller`

HOW backtracking works: we add the current node to path, recurse into neighbors

(which extend the path further), then REMOVE the current node before returning.

This ensures 'path' only ever contains nodes on the CURRENT DFS branch, not nodes

from previously explored branches.

PHASE 3 OPTIMIZE

WHY snapshot path: append list(path) not path, since path is mutated as we backtrack.

Time: O(2^n * n) — exponential paths, linear copy per path. Space: O(n) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _797_AllPathsSourceToTarget:
    def allPathsSourceTarget(self, graph):
        target = len(graph) - 1
        result = []
        def backtrack(node, path):
            if node == target:
                result.append(list(path))
                return
            for neighbor in graph[node]:
                path.append(neighbor)
                backtrack(neighbor, path)
                path.pop()
        backtrack(0, [0])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace allPathsSourceTarget on graph=[[1,2],[3],[3],[]]."

DFS from 0, target=3:

dfs(0,[0]): try 1 and 2.

dfs(1,[0,1]): try 3. dfs(3,[0,1,3]): 3==target -> result.append([0,1,3]). return.

dfs(2,[0,2]): try 3. dfs(3,[0,2,3]): result.append([0,2,3]). return.

result=[[0,1,3],[0,2,3]] ok

WHY DFS with path list: we need all paths (not just shortest) -> DFS explores all branches.

WHY no visited set: acyclic graph (DAG) -> no cycles to trap us.

WHY append copy: path list is mutated during backtracking.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(2^n * n)	**O(n)**
up to 2^n paths, each length <= n	stack + output

For non-DAG graphs, add a visited set to avoid cycles.

Follow-up: All Paths from Source Lead to Destination (#1059) — check if all paths reach dst.

Follow-up: count paths only (not enumerate) — DP on the DAG in O(V+E).